Python Engineering - Semana 03

Semana 03 - Módulos y coordenadas

```
01_python_bootcamp/week_03_modules_and_coordinates/
```

Cómo usar este documento

Este PDF es un índice de repaso de la semana. Cada class resume **qué problema resolvemos, qué conceptos introducimos y qué sintaxis o funciones aparecen**. Si un concepto se te olvida, localízalo aquí y vuelve al ejercicio original.

class_09_modules_imports

EX17 - Movimiento vertical dividido en módulos

Módulo: Archivo .py reutilizable que contiene funciones, clases o constantes.

Arquitectura: vertical_motion.py, data_processing.py y main.py con responsabilidades separadas.

Importaciones: Cada módulo importa directamente lo que utiliza.

main(): Orquesta la aplicación; no debe contener toda la lógica de cálculo.

__name__: if __name__ == '__main__': ejecuta main solo cuando el archivo se lanza directamente.

Diseño: process_trajectory() devuelve datos y evita impresiones internas para mejorar reutilización y pruebas.

class_10_advanced_functions

EX18 - Argumentos avanzados en funciones

Argumentos: posicionales, por palabra clave y con valor por defecto.

***args:** Agrupa argumentos posicionales variables en una tuple.

****kwargs:** Agrupa argumentos variables por palabra clave en un dict.

Desempaquetado: *sequence y **mapping al llamar funciones.

Funciones como objetos: Una función puede pasarse como argumento sin ejecutarla: add frente a add(...).

Idea clave: La firma de una función define cómo recibe información; el desempaquetado permite adaptar estructuras a esa firma.

class_11_coordinate_transforms

EX19 - Matrices de rotación

Matemáticas: Rotaciones 2D y extensión a rotación sobre el eje z en 3D.

NumPy: Producto matriz-vector con @.

Propiedades: R.T @ R = I, R^-1 = R.T para matrices de rotación y det(R)=+1.

Validación: Conservación de norma y uso de np.allclose.

Composición: R(theta2) @ R(theta1) equivale a sumar ángulos para el mismo eje.

Conceptos: rotación activa frente a cambio de marco o convención pasiva.

class_12_engineering_toolkit

EX20 - Conjunto reutilizable de herramientas vectoriales

Funciones: vector_norm(), unit_vector(), angle_between_vectors() y project_vector().

Rotaciones: rotation_matrix_2d_angle(), rotate_2d_vector() y check_rotation_matrix().

NumPy: np.linalg.norm, np.dot, np.clip, np.arccos, np.rad2deg, np.eye, np.linalg.det y np.allclose.

Métodos numéricos: Aplicar np.clip al coseno antes de np.arccos para proteger el dominio [-1, 1].

Validación: Una rotación propia requiere ortogonalidad y determinante +1; una reflexión puede ser ortogonal con determinante -1.

Arquitectura: engineering_toolkit.py contiene las herramientas; main.py crea datos, llama a las funciones y muestra resultados.

Índice rápido de conceptos

Concepto	Dónde verlo
Módulo: Archivo .py reutilizable que contiene funciones, clases o constantes.	class_09_modules_imports
Arquitectura: vertical_motion.py, data_processing.py y main.py con responsabilidades separadas.	class_09_modules_imports
Importaciones: Cada módulo importa directamente lo que utiliza.	class_09_modules_imports
main(): Orquesta la aplicación; no debe contener toda la lógica de cálculo.	class_09_modules_imports
__name__: if __name__ == '__main__': ejecuta main solo cuando el archivo se lanza directamente.	class_09_modules_imports
Diseño: process_trajectory() devuelve datos y evita impresiones internas para mejorar reutilización y pruebas.	class_09_modules_imports
Argumentos: posicionales, por palabra clave y con valor por defecto.	class_10_advanced_functions
*args: Agrupa argumentos posicionales variables en una tuple.	class_10_advanced_functions
**kwargs: Agrupa argumentos variables por palabra clave en un dict.	class_10_advanced_functions
Desempaquetado: *sequence y **mapping al llamar funciones.	class_10_advanced_functions
Funciones como objetos: Una función puede pasarse como argumento sin ejecutarla: add frente a add(...).	class_10_advanced_functions
Idea clave: La firma de una función define cómo recibe información; el desempaquetado permite adaptar estructuras ya existentes.	class_10_advanced_functions
Matemáticas: Rotaciones 2D y extensión a rotación sobre el eje z en 3D.	class_11_coordinate_transforms
NumPy: Producto matriz-vector con @.	class_11_coordinate_transforms
Propiedades: R.T @ R = I, R^-1 = R.T para matrices de rotación y det(R)=+1.	class_11_coordinate_transforms
Validación: Conservación de norma y uso de np.allclose.	class_11_coordinate_transforms
Composición: R(theta2) @ R(theta1) equivale a sumar ángulos para el mismo eje.	class_11_coordinate_transforms
Conceptos: rotación activa frente a cambio de marco o convención pasiva.	class_11_coordinate_transforms
Funciones: vector_norm(), unit_vector(), angle_between_vectors() y project_vector().	class_12_engineering_toolkit

Regla de repaso

Primero busca **el concepto**; después abre el PDF de esa clase; por último ejecuta o modifica el ejercicio. La combinación documento + código es el mecanismo de repaso del curso.